

smart order placing system

Introduction

i-Dine is a web-based, contactless food ordering system for dine-in restaurants. This front-end implementation provides the core user interface for customers to browse the menu, place orders, and interact with the restaurant staff. The system is designed to be intuitive, efficient, and modern.

Feasibility Analysis

The proposed "i-Dine" system is highly feasible from both a technical and operational standpoint.

Type of Feasibility	Description	Analysis for i-Dine
Technical	Determines if the technology is available to support the system.	Yes. The system uses the MERN stack (MongoDB, Express, React.js, Node.js) and Socket.IO, which are widely supported, stable, and well-documented technologies. The QR code and local Wi-Fi integration are standard and easily implemented.
Operational	Evaluates if the system will work effectively in its target environment.	Yes. It is a browser-based web application with a responsive and user-friendly UI. Waitstaff and kitchen personnel can easily use it with minimal training, and its core functionality is intuitive for customers.
Economic	Considers the cost vs. benefits.	Yes. It is cost-effective as it utilizes open-source technologies and reduces the need for physical menus and manual order taking, leading to improved efficiency and resource savings.

Object/Structure-Oriented Analysis (OOA)

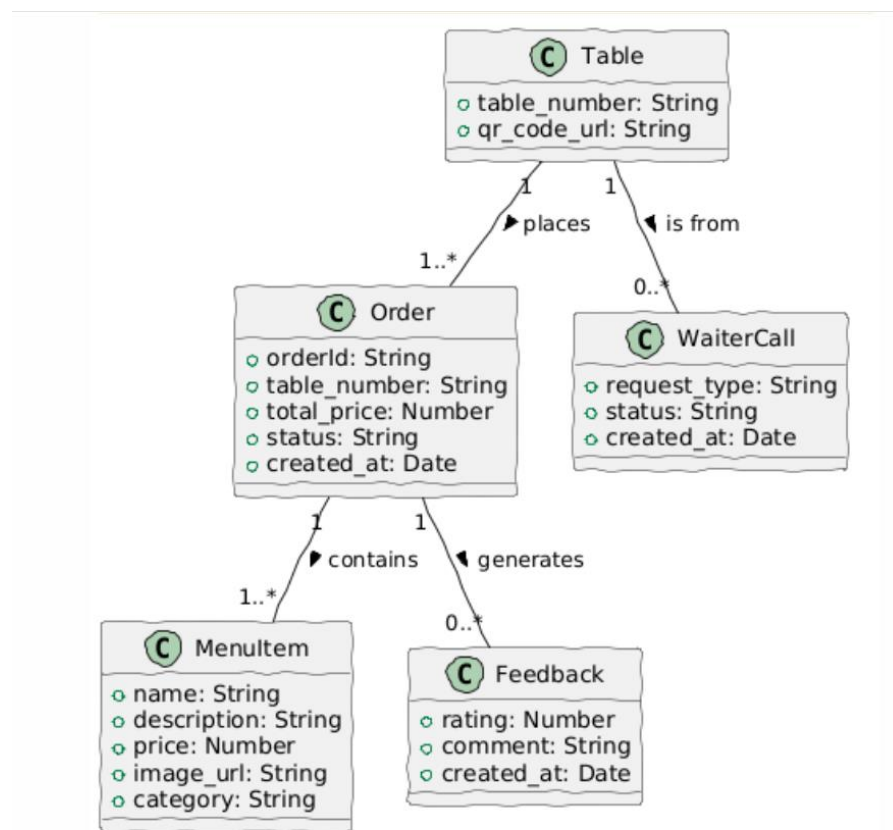
This phase defines the system's key entities and their interactions.

3.1 Key Objects

- **MenuItem**
 - Attributes: id, name, description, price, image_url, category
 - Methods: fetchMenu()
- **Order**
 - Attributes: orderId, tableNumber, items[], totalPrice, status, createdAt
 - Methods: createOrder(), updateStatus(), fetchOrders()

- **Table**
 - Attributes: tableId, tableNumber, qrCodeUrl
- **Feedback**
 - Attributes: feedbackId, orderId, rating, comment, createdAt
 - Methods: submitFeedback()
- **WaiterCall**
 - Attributes: callId, tableNumber, requestType, status, createdAt
 - Methods: callWaiter()

2.Class Diagram:



Software Requirement Specifications (SRS)

This section outlines the functional and non-functional requirements for the system.

Functional Requirements

ID Requirement

- FR1 Scan a unique QR code to access a web-based menu.
- FR2 Display menu items with images, descriptions, and prices.
- FR3 Add, remove, and update items in the cart.
- FR4 Add special instructions for each dish.
- FR5 Submit the order directly to the kitchen system.
- FR6 Provide a "Call Waiter" button for assistance.
- FR7 Allow users to submit feedback and ratings.
- FR8 Display real-time orders on a kitchen dashboard.
- FR9 Display real-time waiter requests on a waiter panel.

Technology Stack:

- Frontend: React.js
- Styling: Custom CSS with a modern, elegant color palette (Beige & Brown).
- Routing: React Router DOM for seamless navigation.
- Assets: Local image assets for the logo and menu items.

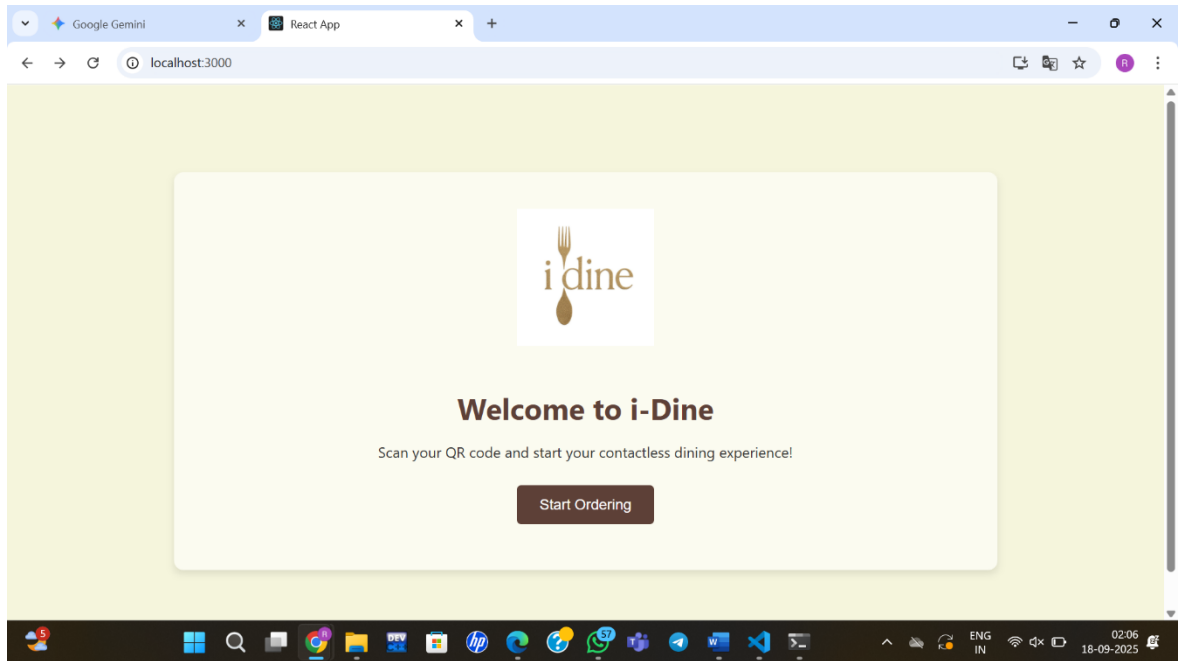
System Architecture

The i-Dine system utilizes a three-tier architecture to separate concerns and ensure a modular design.

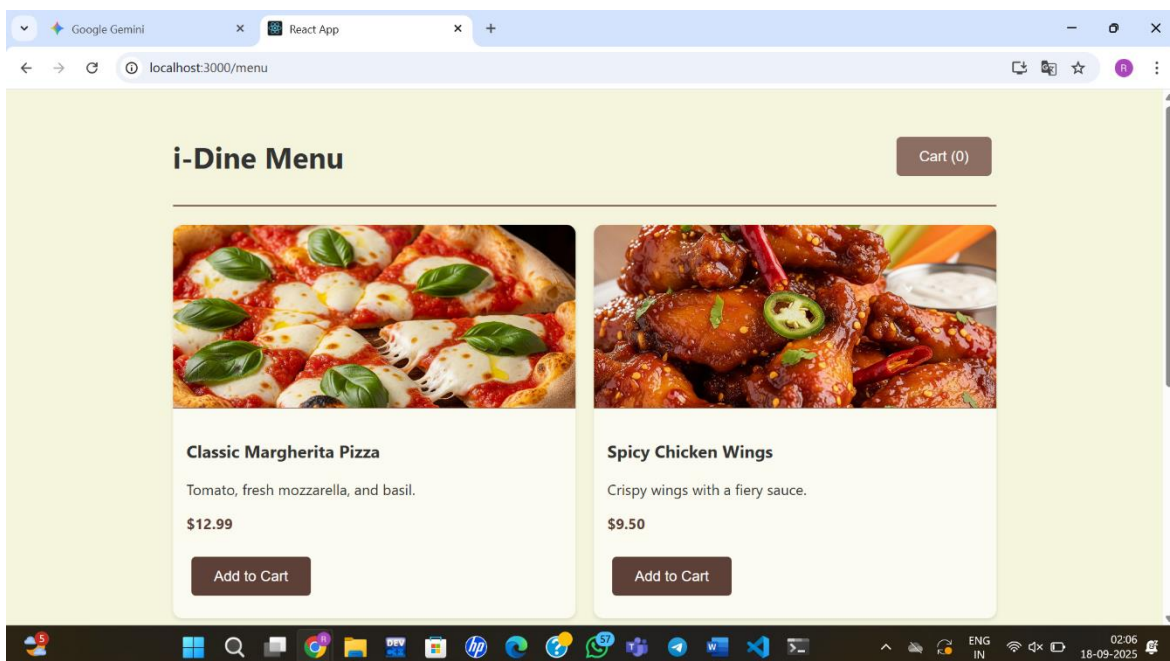
1. Frontend (React.js): Handles the user interface and interactions. This includes the customer-facing menu, cart, and feedback pages, as well as the staff-facing KDS and Waiter Panel.
2. Backend (Node.js + Express): Manages all business logic, API endpoints, and real-time communication. It processes orders, handles data requests, and pushes updates to the frontend.
3. Database (MongoDB): Stores all persistent data, including menu items, orders, feedback, and table information.

Implemented Features (Demo):

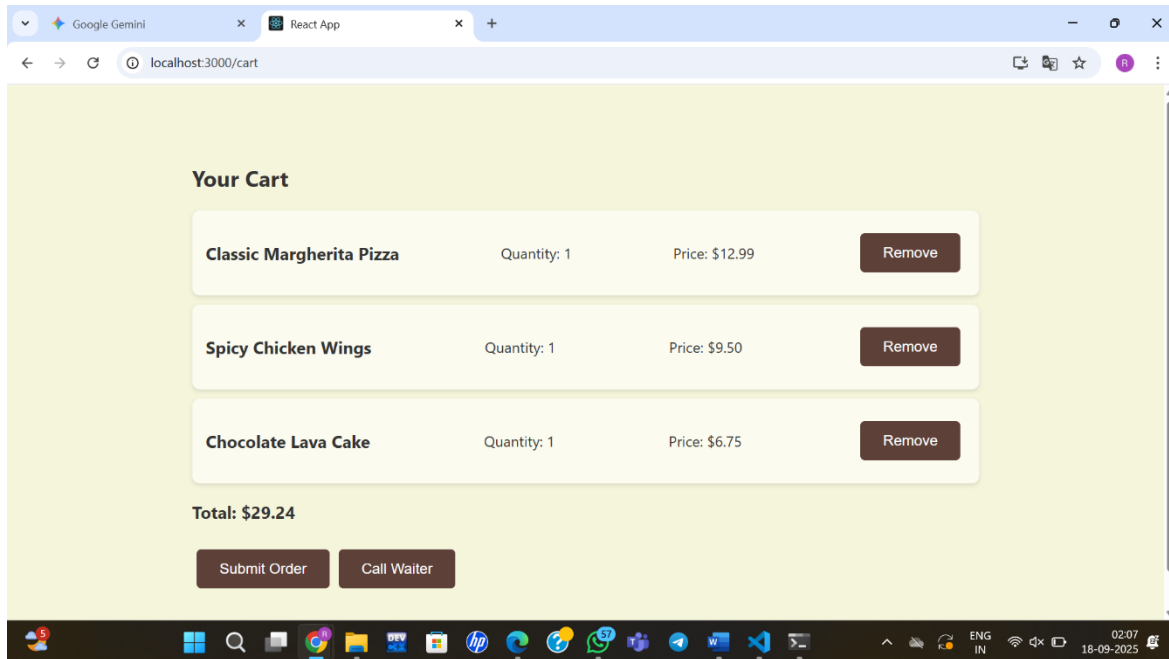
Welcome Page (/): The application's entry point, featuring the project logo and a "Start Ordering" button that directs customers to the menu.



Menu Page (/menu): Displays a categorized menu with item names, descriptions, prices, and images. Customers can add items to their cart from this page.

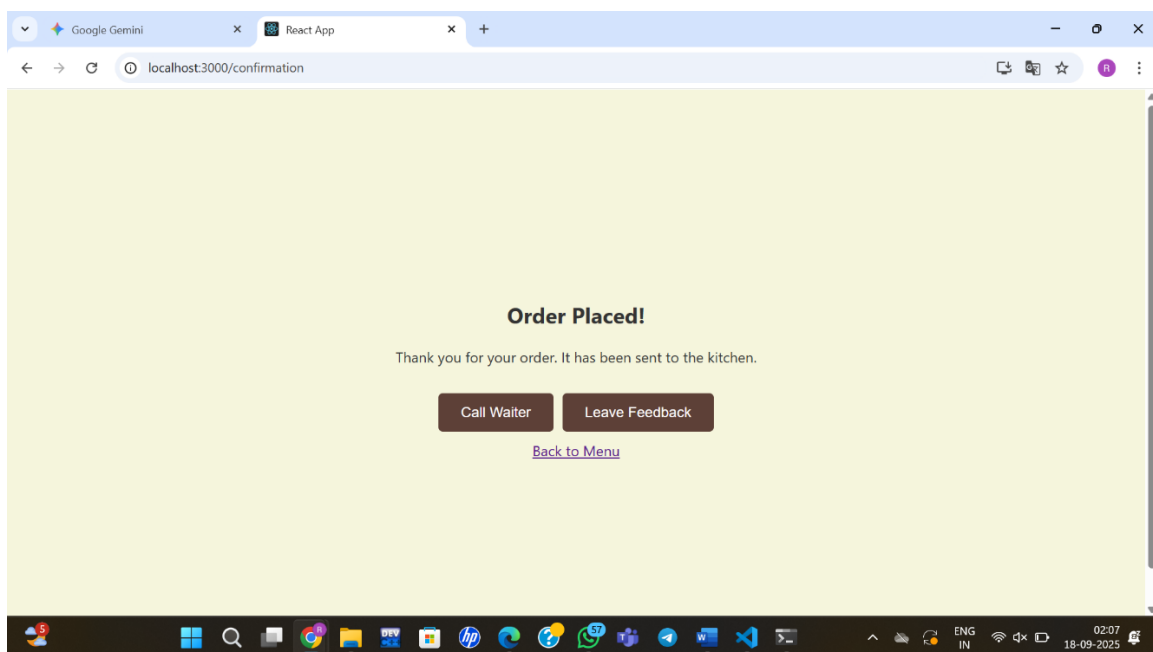


Cart Page (/cart): A dedicated page to review selected items, adjust quantities, and view the total order cost. It includes a "Submit Order" button to finalize the transaction.



Order Confirmation Page (/confirmation): Confirms that the order has been submitted. This page also includes two key features:

- **"Call Waiter" button:** Simulates a real-time request to alert waitstaff.
- **"Leave Feedback" button:** Navigates the user to the feedback section.



Next Steps (Future Implementation) The next phase of development will focus on the staff-facing side of the application to demonstrate the real-time communication functionality.

- **Kitchen Display System (KDS):** A new front-end page will be built to display incoming orders in real time.
- **Waiter Panel:** A page will be created to show live "Call Waiter" requests, allowing staff to acknowledge and manage them.
- **Real-time Integration:** The front end will be integrated with Socket.IO to enable live updates for orders and waiter calls.

Conclusion

The front-end development of the "i-Dine" project has successfully established a modern, responsive, and intuitive customer experience. By leveraging the power of React.js and a modular component-based approach, the foundation is in place for a robust and scalable application. The completed features, from the interactive menu to the real-time communication mock-ups, demonstrate the project's technical feasibility and its potential to revolutionize the dine-in ordering process.

